Planejamento Lógico TaskTracker

Diogo de Oliveira Dias

Bootcamp II — Fase 1: Planejamento Lógico

---

1. Apresentação e Entendimento do Problema

1.1 Identificação

Campo	Informação
Nome completo	Diogo de Oliveira Dias
Curso	Análise e Desenvolvimento de Sistemas
Polo ou turma	Turma B
E-mail institucional	diogo.oliveira07@sempreceub.com
Disciplina	Bootcamp II
Entrega	Fase 1 — Planejamento Lógico
Data	10 de setembro de 2026

1.2 O Problema a Ser Resolvido

Quando uma pessoa ou uma equipe não usa nenhum sistema para organizar o trabalho, as tarefas acabam espalhadas por bilhetes de papel, mensagens de aplicativo, e-mails e, principalmente, pela memória de cada um. Não existe um lugar único que diga o que precisa ser feito. Essa dispersão gera problemas concretos no dia a dia:

> - **Esquecimento.** A tarefa que ninguém anotou simplesmente desaparece, e normalmente só é lembrada depois que o prazo passou.
> - **Priorização pela sensação.** Sem um critério escrito, a pessoa faz primeiro aquilo que lhe parece mais urgente naquele instante, que muitas vezes não é o mais importante.
> - **Perda de prazo silenciosa.** Nada avisa que uma tarefa venceu. A descoberta vem tarde, quando alguém cobra.
> - **Trabalho duplicado.** Em equipe, duas pessoas fazem a mesma coisa porque nenhuma sabia que a outra já havia começado.
> - **Falta de noção do progresso.** No fim da semana ninguém consegue dizer quanto foi realmente concluído, só que trabalhou muito.

Resumindo o problema em uma frase: **a pessoa não consegue responder com segurança a três perguntas básicas sobre o próprio trabalho**, que são o que falta fazer, o que fazer primeiro e o que já está atrasado. É para responder a essas três perguntas que o TaskTracker existe.

1.3 A Solução Proposta (TaskTracker)

O TaskTracker é um sistema de gerenciamento de tarefas que funciona como registro único do que precisa ser feito. Cada tarefa cadastrada recebe um número de identificação próprio e guarda título, descrição, prioridade, categoria e data limite. Na prática, o sistema fará o seguinte pelo usuário:

O sistema permite	Resolve qual dificuldade
Cadastrar uma tarefa com título, prioridade e data limite	Acaba com o esquecimento, porque nada mais depende da memória
Visualizar todas as tarefas pendentes em uma lista única	Dá a visão completa que os bilhetes espalhados não dão
Ordenar a lista automaticamente por prioridade e data limite	Substitui a priorização pela sensação por um critério objetivo
Marcar o que já foi concluído, guardando a data	Separa o que está feito do que falta, sem apagar o histórico
Apontar sozinho as tarefas com prazo vencido	Elimina a descoberta tardia do atraso
Filtrar por situação, prioridade ou categoria	Permite achar rapidamente um subconjunto do trabalho
Mostrar um relatório com quantas tarefas foram concluídas	Torna o progresso visível e mensurável
Guardar tudo em arquivo ao sair e recarregar ao abrir	Garante que o trabalho registrado não se perde entre um uso e outro

Um exemplo de uso: o usuário cadastra a tarefa “Entregar relatório” com prioridade Alta e data limite 12 de setembro. Ao abrir o sistema no dia seguinte, essa tarefa aparece no topo da lista, acima de uma tarefa de prioridade MÉDIA cadastrada antes. Depois de entregar o relatório, ele marca a tarefa como concluída e o sistema registra sozinho a data da conclusão. Se o dia 12 passar sem a conclusão, a tarefa passa a aparecer sinalizada como atrasada.

---

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de Dados

São todas as informações que o usuário precisará digitar no sistema.

2.1.1 Entradas do cadastro de uma tarefa

Entrada	Tipo	Obrigatório	Valores aceitos	Exemplo
Título da tarefa	Texto	Sim	De 3 a 100 caracteres	Entregar relatório
Descrição	Texto	Não	Até 500 caracteres	Consolidar os números de agosto e revisar
Prioridade	Texto	Sim	Alta, Média ou Baixa	Alta
Categoria	Texto	Não	Livre	Trabalho
Data limite	Data	Não	Formato dia, mês e ano; não pode ser anterior a hoje	12/09/2026

2.1.2 Entradas das demais operações

Entrada	Tipo	Usada em	Valores aceitos	Exemplo
Opção do menu	Número inteiro	Todas as operações	De 0 a 11	1
Identificador da tarefa	Número inteiro	Buscar, editar, iniciar, concluir, cancelar e excluir	Qualquer identificador existente	3
Campo a alterar	Número inteiro	Edição de tarefa	De 1 a 5	3
Novo valor do campo	Texto ou data	Edição de tarefa	Mesmas regras do cadastro	Média
Confirmação de exclusão	Texto	Exclusão de tarefa	A palavra SIM confirma; qualquer outra coisa cancela	SIM
Situação para filtro	Texto	Filtragem	Pendente, Em andamento, Concluída ou Cancelada	Pendente

2.2 Saídas de Dados

É tudo o que o sistema mostrará na tela para o usuário.

Saída	Quando aparece	Exemplo do que será exibido
Menu de opções	No início e após cada operação	Lista numerada de 0 a 11 com as ações disponíveis
Mensagem de confirmação de cadastro	Após cadastrar uma tarefa	Tarefa cadastrada com o identificador 1
Lista de tarefas organizada	Ao pedir a listagem	Identificador, título, prioridade, situação, data limite e indicação de atraso, ordenados da prioridade mais alta para a mais baixa
Detalhe de uma tarefa	Ao buscar por identificador	Todos os campos da tarefa encontrada
Mensagem de erro de validação	Quando o dado digitado é inválido	ERRO: a prioridade deve ser Alta, Média ou Baixa
Mensagem de tarefa inexistente	Quando o identificador informado não existe	ERRO: tarefa não encontrada
Confirmação de alteração de situação	Ao iniciar, concluir ou cancelar	Situação atualizada para Concluída
Lista de tarefas atrasadas	Ao consultar pendências vencidas	Identificador, título e quantos dias de atraso cada tarefa acumula
Resultado do filtro	Ao filtrar por situação	As tarefas encontradas e a quantidade total
Total de tarefas concluídas	No relatório	Quantidade por situação, taxa de conclusão e tempo médio de execução
Mensagem de encerramento	Ao sair do sistema	Dados gravados e sistema encerrado

2.3 Regras do Sistema

São as condições que o sistema deve respeitar sempre, independentemente do que o usuário digitar.

Código	Regra	Por que existe
R1	O sistema não pode aceitar tarefas sem título. O título deve ter de 3 a 100 caracteres.	Uma tarefa sem nome não pode ser identificada depois pelo usuário
R2	A prioridade deve ser apenas Alta, Média ou Baixa.	Garante que a ordenação tenha sempre um critério comparável
R3	A data limite não pode ser anterior à data de criação da tarefa.	Uma tarefa não pode nascer já vencida
R4	Toda tarefa cadastrada começa com a situação Pendente.	Padroniza o ponto de partida do ciclo de vida
R5	Cada tarefa recebe um identificador único gerado pelo sistema, nunca digitado pelo usuário.	Evita que duas tarefas tenham o mesmo número
R6	O identificador nunca é reaproveitado, mesmo depois que a tarefa é excluída.	Impede que um número antigo aponte para uma tarefa diferente
R7	Uma tarefa concluída ou cancelada não pode ser editada, apenas consultada ou excluída.	Preserva o histórico do que realmente aconteceu
R8	A data de conclusão é preenchida pelo sistema no momento em que a tarefa é concluída.	Evita que o usuário informe uma data de conclusão falsa
R9	Uma tarefa é considerada atrasada quando a data limite já passou e a situação não é Concluída nem Cancelada.	Define objetivamente o que conta como atraso
R10	A exclusão só acontece se o usuário confirmar digitando SIM, e não pode ser desfeita.	Protege contra a perda acidental de uma tarefa

---

3. Receita Lógica do Sistema (Algoritmo em Português)

Abaixo está o passo a passo lógico de como o sistema funcionará, escrito como uma receita, em linguagem simples, de modo que qualquer pessoa consiga acompanhar mesmo sem saber programar.

3.1 Passo a Passo do Cadastro

1. Pedir ao usuário o título da tarefa.
2. Verificar se o título não está vazio. Se estiver vazio ou muito curto, avisar o erro e pedir de novo.
3. Pedir a descrição da tarefa, que pode ficar em branco.
4. Pedir a prioridade. Se não for Alta, Média ou Baixa, avisar o erro e pedir de novo.
5. Pedir a categoria, que também pode ficar em branco.
6. Pedir a data limite. Se for uma data que já passou, avisar o erro e pedir de novo.
7. Gerar um número de identificação novo para a tarefa.
8. Marcar a tarefa como Pendente e guardar a data de hoje como data de criação.
9. Salvar a tarefa na lista de tarefas.
10. Exibir mensagem de sucesso informando o número gerado.

3.2 Passo a Passo da Visualização

1. Acessar a lista de tarefas.
2. Verificar se existe alguma tarefa cadastrada.
3. Se a lista estiver vazia, avisar o usuário que não há nada para mostrar e encerrar.
4. Se houver tarefas, colocar a lista em ordem: primeiro as de prioridade mais alta e, entre as de mesma prioridade, primeiro as de data limite mais próxima.
5. Percorrer a lista mostrando uma tarefa por vez na tela.
6. Para cada tarefa, exibir o número, o título, a prioridade, a situação e a data limite.
7. Verificar se a data limite já passou e a tarefa ainda não foi concluída. Se for o caso, marcar como atrasada.
8. No final, informar quantas tarefas foram exibidas.

3.3 Passo a Passo da Conclusão

1. Pedir ao usuário o número da tarefa que ele quer concluir.
2. Procurar esse número na lista de tarefas.
3. Se não encontrar, avisar que a tarefa não existe e encerrar.
4. Se a tarefa já estiver concluída ou cancelada, avisar que ela já foi encerrada e não pode mudar.
5. Caso contrário, mudar a situação da tarefa para Concluída.
6. Guardar a data de hoje como data de conclusão.
7. Salvar a alteração e confirmar ao usuário.

3.4 Passo a Passo da Exclusão

1. Pedir ao usuário o número da tarefa que ele quer excluir.
2. Procurar esse número na lista.
3. Se não encontrar, avisar que a tarefa não existe e encerrar.
4. Mostrar o título da tarefa e avisar que a exclusão não pode ser desfeita.
5. Pedir que o usuário digite SIM para confirmar.
6. Se ele digitar qualquer outra coisa, cancelar a operação e encerrar.
7. Se confirmar, retirar a tarefa da lista.
8. Deixar o contador de números como estava, para que o número da tarefa excluída nunca seja reaproveitado.
9. Salvar e confirmar ao usuário.

3.5 Passo a Passo do Programa Inteiro

1. Abrir o programa.
2. Ler o arquivo onde as tarefas ficam guardadas e carregar tudo para a memória.
3. Descobrir qual foi o maior número de identificação já usado, para continuar a contagem a partir dele.
4. Mostrar o menu com as opções disponíveis.
5. Ler a opção que o usuário escolheu.
6. Executar a ação correspondente. Se a opção não existir, avisar o erro.
7. Voltar a mostrar o menu, até que o usuário escolha sair.
8. Ao sair, gravar todas as tarefas de volta no arquivo.
9. Fechar o programa.

3.6 Como o sistema decide a ordem das tarefas

Para colocar a lista em ordem, o sistema precisa comparar prioridades que são palavras, e não números. A saída é dar um peso a cada uma:

Prioridade	Peso
Alta	3
Média	2
Baixa	1

Com os pesos definidos, a receita da ordenação fica assim:

> 1. Começar pela segunda tarefa da lista.
> 2. Guardar essa tarefa de lado, como a que precisa achar seu lugar.
> 3. Comparar a tarefa guardada com as anteriores, uma a uma, de trás para frente.
> 4. Enquanto a tarefa anterior tiver peso menor, ou tiver o mesmo peso mas data limite mais distante, empurrar essa anterior uma posição para a direita.
> 5. Colocar a tarefa guardada na posição que ficou livre.
> 6. Passar para a próxima tarefa e repetir até o fim da lista.

3.7 O ciclo de vida de uma tarefa

Uma tarefa não pode pular de qualquer situação para qualquer outra. Os caminhos permitidos são estes:

Situação atual	Ação	Nova situação
Pendente	Iniciar	Em andamento
Pendente	Concluir	Concluída
Pendente	Cancelar	Cancelada
Em andamento	Concluir	Concluída
Em andamento	Cancelar	Cancelada
Concluída	Nenhuma	Situação final
Cancelada	Nenhuma	Situação final

Concluída e Cancelada são pontos sem volta. É isso que garante que o histórico do que aconteceu não possa ser reescrito depois.

3.8 Pseudocódigo

A receita acima, escrita agora em pseudocódigo. Cada validação traz em comentário a regra da seção 2.3 que ela aplica.

3.8.1 Estruturas de dados

```
CONSTANTE MAX_TAREFAS = 1000

REGISTRO Tarefa
    inteiro   id
    cadeia    titulo
    cadeia    descricao
    cadeia    prioridade        // "ALTA" | "MEDIA" | "BAIXA"
    cadeia    situacao          // "PENDENTE" | "EM_ANDAMENTO" | "CONCLUIDA" | "CANCELADA"
    cadeia    categoria
    data      data_criacao
    data      data_limite
    data      data_conclusao
FIM_REGISTRO

VETOR   tarefas[1..MAX_TAREFAS] DO TIPO Tarefa
INTEIRO total     <- 0      // quantas tarefas existem agora
INTEIRO proximoId <- 1      // contador de identificadores, nunca reutilizado
CADEIA  ARQUIVO   <- "tasktracker.json"
```

3.8.2 Programa principal

```
ALGORITMO TaskTracker

INICIO
    INTEIRO opcao

    carregarDados()
```

```
    REPITA
        ESCREVA "1  - Cadastrar tarefa"
        ESCREVA "2  - Listar todas as tarefas"
        ESCREVA "3  - Buscar tarefa por identificador"
        ESCREVA "4  - Editar tarefa"
        ESCREVA "5  - Iniciar tarefa"
        ESCREVA "6  - Concluir tarefa"
        ESCREVA "7  - Cancelar tarefa"
        ESCREVA "8  - Excluir tarefa"
        ESCREVA "9  - Filtrar por situacao"
        ESCREVA "10 - Listar tarefas atrasadas"
        ESCREVA "11 - Relatorio de produtividade"
        ESCREVA "0  - Sair"
        ESCREVA "Escolha uma opcao: "
        LEIA opcao

        ESCOLHA opcao
            CASO 1  : cadastrarTarefa()
            CASO 2  : listarTarefas()
            CASO 3  : buscarTarefa()
            CASO 4  : editarTarefa()
            CASO 5  : mudarSituacao("EM_ANDAMENTO")
            CASO 6  : mudarSituacao("CONCLUIDA")
            CASO 7  : mudarSituacao("CANCELADA")
            CASO 8  : excluirTarefa()
            CASO 9  : filtrarPorSituacao()
            CASO 10 : listarAtrasadas()
            CASO 11 : gerarRelatorio()
            CASO 0  : ESCREVA "Encerrando o sistema."
            SENAO   : ESCREVA "ERRO: opcao invalida."
        FIM_ESCOLHA

    ATE opcao = 0

    salvarDados()
FIM
```

3.8.3 Cadastrar tarefa

```
PROCEDIMENTO cadastrarTarefa()
    Tarefa nova

    SE total >= MAX_TAREFAS ENTAO
        ESCREVA "ERRO: base cheia, nao e possivel cadastrar."
        RETORNE
    FIM_SE

    // --- titulo obrigatorio, de 3 a 100 caracteres (R1)
    REPITA
        ESCREVA "Titulo: "
        LEIA nova.titulo
        SE comprimento(nova.titulo) < 3 OU comprimento(nova.titulo) > 100 ENTAO
            ESCREVA "ERRO: o titulo e obrigatorio e deve ter de 3 a 100 caracteres."
        FIM_SE
    ATE comprimento(nova.titulo) >= 3 E comprimento(nova.titulo) <= 100

    ESCREVA "Descricao (opcional): "
    LEIA nova.descricao

    // --- prioridade somente Alta, Media ou Baixa (R2)
    REPITA
        ESCREVA "Prioridade [ALTA / MEDIA / BAIXA]: "
        LEIA nova.prioridade
        SE nova.prioridade <> "ALTA"
           E nova.prioridade <> "MEDIA"
           E nova.prioridade <> "BAIXA" ENTAO
            ESCREVA "ERRO: a prioridade deve ser Alta, Media ou Baixa."
        FIM_SE
    ATE nova.prioridade = "ALTA" OU nova.prioridade = "MEDIA"
        OU nova.prioridade = "BAIXA"

    ESCREVA "Categoria (opcional): "
    LEIA nova.categoria

    // --- data limite nao pode ser anterior a hoje (R3)
    REPITA
        ESCREVA "Data limite [dd/mm/aaaa]: "
        LEIA nova.data_limite
        SE nova.data_limite < dataAtual() ENTAO
            ESCREVA "ERRO: a data limite nao pode ser anterior a hoje."
        FIM_SE
    ATE nova.data_limite >= dataAtual()

    // --- campos que so o sistema preenche (R4, R5)
    nova.id             <- proximoId
    nova.situacao       <- "PENDENTE"
    nova.data_criacao   <- dataAtual()
    nova.data_conclusao <- NULO
```

```
    total          <- total + 1
    tarefas[total] <- nova
    proximoId      <- proximoId + 1

    salvarDados()
    ESCREVA "Tarefa cadastrada com o identificador ", nova.id
FIM_PROCEDIMENTO
```

3.8.4 Ordenar e listar

```
// Transforma o texto da prioridade em numero, para permitir comparacao.
FUNCAO pesoPrioridade(cadeia p) : inteiro
    SE p = "ALTA"  ENTAO RETORNE 3
    SE p = "MEDIA" ENTAO RETORNE 2
    SE p = "BAIXA" ENTAO RETORNE 1
    RETORNE 0
FIM_FUNCAO

// Ordenacao por insercao.
// Criterio principal : prioridade da mais alta para a mais baixa.
// Desempate          : data limite mais proxima primeiro.
PROCEDIMENTO ordenarTarefas()
    INTEIRO i, j
    Tarefa  atual

    PARA i DE 2 ATE total FACA
        atual <- tarefas[i]
        j     <- i - 1

        ENQUANTO j >= 1 E (
                 pesoPrioridade(tarefas[j].prioridade) < pesoPrioridade(atual.prioridade)
                 OU (pesoPrioridade(tarefas[j].prioridade) = pesoPrioridade(atual.prioridade)
                     E tarefas[j].data_limite > atual.data_limite)
               ) FACA
            tarefas[j + 1] <- tarefas[j]
            j <- j - 1
        FIM_ENQUANTO

        tarefas[j + 1] <- atual
    FIM_PARA
FIM_PROCEDIMENTO

PROCEDIMENTO listarTarefas()
    INTEIRO i

    SE total = 0 ENTAO
        ESCREVA "Nenhuma tarefa cadastrada."
        RETORNE
    FIM_SE

    ordenarTarefas()

    ESCREVA "ID | TITULO | PRIORIDADE | SITUACAO | DATA LIMITE | ATRASO"
    PARA i DE 1 ATE total FACA
        ESCREVA tarefas[i].id, " | ",
                tarefas[i].titulo, " | ",
                tarefas[i].prioridade, " | ",
                tarefas[i].situacao, " | ",
                tarefas[i].data_limite, " | ",
                SE estaAtrasada(tarefas[i]) ENTAO "ATRASADA" SENAO "EM DIA"
    FIM_PARA

    ESCREVA "Total exibido: ", total, " tarefa(s)."
FIM_PROCEDIMENTO
```

3.8.5 Buscar, mudar situação e excluir

```
// Busca sequencial: devolve a posicao no vetor, ou -1 se nao existir.
FUNCAO buscarIndicePorId(inteiro idProcurado) : inteiro
    INTEIRO i
    PARA i DE 1 ATE total FACA
        SE tarefas[i].id = idProcurado ENTAO
            RETORNE i
        FIM_SE
    FIM_PARA
    RETORNE -1
FIM_FUNCAO

PROCEDIMENTO mudarSituacao(cadeia novaSituacao)
    INTEIRO id, indice

    ESCREVA "Identificador da tarefa: "
    LEIA id
    indice <- buscarIndicePorId(id)

    SE indice = -1 ENTAO
        ESCREVA "ERRO: tarefa nao encontrada."
```

```
        RETORNE
    FIM_SE

    // Situacoes finais nao mudam mais (R7)
    SE tarefas[indice].situacao = "CONCLUIDA"
       OU tarefas[indice].situacao = "CANCELADA" ENTAO
        ESCREVA "ERRO: a tarefa ja esta encerrada e nao pode ser alterada."
        RETORNE
    FIM_SE

    SE NAO transicaoValida(tarefas[indice].situacao, novaSituacao) ENTAO
        ESCREVA "ERRO: mudanca invalida."
        RETORNE
    FIM_SE

    tarefas[indice].situacao <- novaSituacao

    SE novaSituacao = "CONCLUIDA" ENTAO
        tarefas[indice].data_conclusao <- dataAtual()      // R8
    FIM_SE

    salvarDados()
    ESCREVA "Situacao atualizada para ", novaSituacao
FIM_PROCEDIMENTO

FUNCAO transicaoValida(cadeia origem, cadeia destino) : logico
    SE origem = "PENDENTE"     E destino = "EM_ANDAMENTO" ENTAO RETORNE VERDADEIRO
    SE origem = "PENDENTE"     E destino = "CONCLUIDA"    ENTAO RETORNE VERDADEIRO
    SE origem = "PENDENTE"     E destino = "CANCELADA"    ENTAO RETORNE VERDADEIRO
    SE origem = "EM_ANDAMENTO" E destino = "CONCLUIDA"    ENTAO RETORNE VERDADEIRO
    SE origem = "EM_ANDAMENTO" E destino = "CANCELADA"    ENTAO RETORNE VERDADEIRO
    RETORNE FALSO
FIM_FUNCAO

PROCEDIMENTO excluirTarefa()
    INTEIRO id, indice, i
    CADEIA  confirmacao

    ESCREVA "Identificador da tarefa: "
    LEIA id
    indice <- buscarIndicePorId(id)

    SE indice = -1 ENTAO
        ESCREVA "ERRO: tarefa nao encontrada."
        RETORNE
    FIM_SE

    // Exclusao exige confirmacao e nao pode ser desfeita (R10)
    ESCREVA "Excluir a tarefa '", tarefas[indice].titulo, "'?"
    ESCREVA "Esta acao nao pode ser desfeita. Digite SIM para confirmar: "
    LEIA confirmacao

    SE confirmacao <> "SIM" ENTAO
        ESCREVA "Operacao cancelada."
        RETORNE
    FIM_SE

    PARA i DE indice ATE total - 1 FACA
        tarefas[i] <- tarefas[i + 1]
    FIM_PARA

    total <- total - 1
    // proximoId nao diminui: identificador nunca se repete (R6)

    salvarDados()
    ESCREVA "Tarefa excluida."
FIM_PROCEDIMENTO
```

3.8.6 Tarefas atrasadas, relatório e persistência

```
// Aplica a regra R9
FUNCAO estaAtrasada(Tarefa t) : logico
    SE t.situacao <> "CONCLUIDA"
       E t.situacao <> "CANCELADA"
       E t.data_limite < dataAtual() ENTAO
        RETORNE VERDADEIRO
    SENAO
        RETORNE FALSO
    FIM_SE
FIM_FUNCAO

PROCEDIMENTO gerarRelatorio()
    INTEIRO i, qtdPendente, qtdAndamento, qtdConcluida, qtdCancelada, qtdAtrasada
    REAL    taxaConclusao

    qtdPendente <- 0   qtdAndamento <- 0   qtdConcluida <- 0
    qtdCancelada <- 0  qtdAtrasada <- 0

    PARA i DE 1 ATE total FACA
```

```
        ESCOLHA tarefas[i].situacao
            CASO "PENDENTE"     : qtdPendente  <- qtdPendente + 1
            CASO "EM_ANDAMENTO" : qtdAndamento <- qtdAndamento + 1
            CASO "CONCLUIDA"    : qtdConcluida <- qtdConcluida + 1
            CASO "CANCELADA"    : qtdCancelada <- qtdCancelada + 1
        FIM_ESCOLHA

        SE estaAtrasada(tarefas[i]) ENTAO
            qtdAtrasada <- qtdAtrasada + 1
        FIM_SE
    FIM_PARA

    SE total > 0 ENTAO
        taxaConclusao <- (qtdConcluida / total) * 100
    SENAO
        taxaConclusao <- 0
    FIM_SE

    ESCREVA "Total de tarefas .......: ", total
    ESCREVA "Pendentes ..............: ", qtdPendente
    ESCREVA "Em andamento ...........: ", qtdAndamento
    ESCREVA "Concluidas .............: ", qtdConcluida
    ESCREVA "Canceladas .............: ", qtdCancelada
    ESCREVA "Atrasadas ..............: ", qtdAtrasada
    ESCREVA "Taxa de conclusao ......: ", taxaConclusao, "%"
FIM_PROCEDIMENTO

PROCEDIMENTO carregarDados()
    INTEIRO maiorId, i

    SE NAO arquivoExiste(ARQUIVO) ENTAO
        total <- 0
        proximoId <- 1
        RETORNE
    FIM_SE

    abrirArquivo(ARQUIVO, "leitura")
    total <- lerInteiro()
    PARA i DE 1 ATE total FACA
        tarefas[i] <- lerRegistroTarefa()
    FIM_PARA
    fecharArquivo()

    // Recalcula o contador para nunca repetir identificador (R6)
    maiorId <- 0
    PARA i DE 1 ATE total FACA
        SE tarefas[i].id > maiorId ENTAO
            maiorId <- tarefas[i].id
        FIM_SE
    FIM_PARA
    proximoId <- maiorId + 1
FIM_PROCEDIMENTO

PROCEDIMENTO salvarDados()
    INTEIRO i

    // Grava num arquivo temporario e so entao substitui o original,
    // para que uma falha no meio da gravacao nao estrague a base.
    abrirArquivo("temporario.json", "escrita")
    escreverInteiro(total)
    PARA i DE 1 ATE total FACA
        escreverRegistroTarefa(tarefas[i])
    FIM_PARA
    fecharArquivo()
    renomearArquivo("temporario.json", ARQUIVO)
FIM_PROCEDIMENTO
```

3.9 Teste de Mesa

Simulação passo a passo da execução, considerando a data de hoje como 10 de setembro de 2026 e a lista começando vazia.

Passo	O que o usuário faz	total	proximoId	O que o sistema responde
1	Cadastra “Entregar relatório”, prioridade Alta, data limite 12/09/2026	1	2	Tarefa cadastrada com o identificador 1
2	Tenta cadastrar com o título “Ok”	1	2	ERRO: o título é obrigatório e deve ter de 3 a 100 caracteres
3	Corrige para “Revisar código”, prioridade Média, data limite 20/09/2026	2	3	Tarefa cadastrada com o identificador 2
4	Tenta cadastrar “Estudar SQL” com prioridade Urgente	2	3	ERRO: a prioridade deve ser Alta, Média ou Baixa
5	Corrige para Alta, mas informa data limite 01/09/2026	2	3	ERRO: a data limite não pode ser anterior a hoje
6	Corrige a data limite para 15/09/2026	3	4	Tarefa cadastrada com o identificador 3
7	Pede a listagem	3	4	Exibe na ordem: id 1 (Alta, 12/09), id 3 (Alta, 15/09), id 2 (Média, 20/09)
8	Conclui a tarefa 1	3	4	Situação atualizada para Concluída, data de conclusão registrada
9	Tenta editar a tarefa 1	3	4	ERRO: tarefa encerrada não pode ser editada
10	Exclui a tarefa 2 e confirma com SIM	2	4	Tarefa excluída, mas o contador continua em 4
11	Pede o relatório	2	4	Total 2, concluídas 1, pendentes 1, taxa de conclusão 50 por cento
12	Sai do sistema	2	4	Dados gravados e sistema encerrado

Cinco passos comprovam regras específicas. O passo 2 comprova a regra R1. O passo 4 comprova a R2 e o passo 5 comprova a R3. O passo 7 mostra o desempate por data limite, já que as tarefas 1 e 3 têm a mesma prioridade e a de prazo mais próximo aparece primeiro. O passo 9 comprova a R7 e o passo 10 comprova a R6, porque o contador não volta atrás depois da exclusão.

3.10 Validação do algoritmo

Para conferir que a receita não tem falha de lógica, ela foi traduzida linha a linha para uma linguagem de programação e executada com exatamente o cenário do teste de mesa. Esta foi a saída real:

```
PASSO | ACAO                              | total | proxId | SAIDA
-----------------------------------------------------------------------------------------
    1 | cadastrar 'Entregar relatorio' Alta |     1 |      2 | Tarefa cadastrada, id 1
    2 | cadastrar titulo 'Ok'               |     1 |      2 | ERRO: titulo de 3 a 100
    3 | corrige para 'Revisar codigo' Media |     2 |      3 | Tarefa cadastrada, id 2
    4 | cadastrar prioridade 'Urgente'      |     2 |      3 | ERRO: Alta, Media ou Baixa
    5 | corrige Alta, data limite 01/09     |     2 |      3 | ERRO: data limite passada
    6 | corrige data limite 15/09           |     3 |      4 | Tarefa cadastrada, id 3
    7 | listar (ordenado)                   |     3 |      4 | id1(Alta,12/09) id3(Alta,15/09) id2(Media,20/09)
    8 | concluir tarefa 1                   |     3 |      4 | Situacao atualizada para CONCLUIDA
    9 | editar tarefa 1                     |     3 |      4 | ERRO: tarefa encerrada
   10 | excluir tarefa 2, confirma SIM      |     2 |      4 | Tarefa excluida
   11 | relatorio                           |     2 |      4 | total=2 concluidas=1 taxa=50%
   12 | sair                                |     2 |      4 | Dados gravados
-----------------------------------------------------------------------------------------

VERIFICACOES EXTRAS:
  ordem esperada id1, id3, id2 ............... confirmada
```

```
  proximoId apos a exclusao (deve ser 4) ..... 4
  taxa de conclusao (deve ser 50%) ........... 50%
  tentar reabrir uma tarefa concluida ........ recusado, como previsto
  buscar identificador inexistente ........... devolveu -1, como previsto
  excluir sem digitar SIM .................... operacao cancelada, como previsto
```

Os doze passos coincidiram com o teste de mesa, e as seis verificações extras confirmaram as regras R2, R6, R7 e R10. A lógica está consistente e pronta para virar código na Fase 2.

---

4. Mapeamento Arquitetural para Git e GitHub (Preparação para a Fase 2)

4.1 Desenho da Árvore de Repositório

Representação em texto da estrutura de arquivos que será enviada ao GitHub na Fase 2.

```
tasktracker-projeto/
|
+-- README.md                    # Apresentacao do projeto no GitHub
+-- .gitignore                   # Arquivo para ignorar arquivos temporarios
|
+-- docs/
|   +-- planejamento_logico.pdf  # Este planejamento da Fase 1
|
+-- src/
    +-- main.py                  # Arquivo principal do programa Python
```

4.1.1 Para que serve cada arquivo

Arquivo ou pasta	Finalidade
README.md	Primeira coisa que aparece ao abrir o repositório no GitHub. Vai explicar o que é o TaskTracker, como executar e quais funcionalidades existem.
.gitignore	Lista dos arquivos que o Git deve ignorar, como arquivos temporários e a base de dados local, que não precisam ir para o repositório.
docs/	Guarda a documentação do projeto. Começa recebendo este planejamento da Fase 1, exportado em PDF.
src/	Guarda o código-fonte. Separar o código da documentação deixa o repositório organizado desde o início.
src/main.py	Arquivo principal do programa. É ele que será executado e que conterá o laço do menu e as sub-rotinas descritas na seção 3.8.

4.2 Plano de Aprendizado para Git e GitHub

Na Fase 2 espero entender, na prática, o que realmente acontece quando faço um commit, deixando de tratar o Git como uma sequência de comandos decorados para entendê-lo como um histórico de versões do projeto. Quero aprender a salvar alterações de forma organizada, com mensagens de commit que descrevam o que mudou e por quê, de modo que qualquer pessoa consiga acompanhar a evolução do TaskTracker apenas lendo o histórico. Também pretendo aprender a criar branches para desenvolver novas funcionalidades sem quebrar o que já funciona, e a lidar com conflitos de código quando eles aparecerem, entendendo por que surgem em vez de apenas contorná-los. Por fim, quero saber publicar o projeto no GitHub e usá-lo como portfólio, aproveitando o repositório não só para guardar o código, mas para mostrar o processo de construção, desde este planejamento até a versão final do sistema.

4.2.1 Tópicos que pretendo dominar

Tópico	O que espero conseguir fazer
Entender o que é um commit	Saber o que fica gravado em cada commit e escrever mensagens que expliquem a mudança
Salvar alterações de forma organizada	Agrupar mudanças relacionadas em um mesmo commit, em vez de enviar tudo de uma vez
Criar branches para novas funcionalidades	Desenvolver uma funcionalidade isolada e só juntá-la ao projeto quando estiver pronta
Resolver conflitos de código	Entender por que o conflito aconteceu e escolher conscientemente qual versão manter
Publicar o projeto no GitHub	Enviar o repositório local para a nuvem e mantê-lo atualizado
Trabalhar com histórico de versões	Consultar o que mudou entre duas versões e voltar atrás quando necessário

4.3 Sequência de commits planejada

Ordem pretendida para os primeiros commits, de modo que a cada envio o projeto continue funcionando.

Ordem	Mensagem do commit	O que entra
1	Estrutura inicial do projeto	Pastas, README e arquivo de exclusões do Git
2	Adiciona planejamento lógico da Fase 1	Este documento exportado para a pasta de documentação
3	Implementa cadastro de tarefas	Cadastro com as validações R1, R2 e R3
4	Implementa listagem e ordenação	Listagem ordenada por prioridade e data limite
5	Implementa mudança de situação	Iniciar, concluir e cancelar, respeitando o ciclo de vida
6	Implementa exclusão com confirmação	Exclusão seguindo a regra R10
7	Adiciona persistência em arquivo	Carregar e salvar a lista de tarefas

---

Importante: este documento será a referência principal para o desenvolvimento do projeto nas próximas fases da disciplina.